

恶意代码分析与防治技术实验报告

Lab 13



网络空间安全学院

信息安全专业

2211985 李佳璐

一、实验目的

完成课本Lab13的实验内容，编写Yara规则，并尝试IDA Python的自动化分析

二、实验过程

Lab13-01

1.过程分析

- 静态分析

使用strings工具查看字符串列表

```
Mozilla/4.0
http://%s/%s/
Could not load exe.
Could not locate dialog box.
Could not load dialog box.
Could not lock dialog box.
%02x
'n
'n
lSe
<Se
TR
LLL
KIZKORXZWUZWLZI^ZUZWBRH
```

可以看到 http 请求的格式化字符串，以及一个 user-agent 字符串。但是没有看到网址信息。

同时，还可以看到一个编码表，包含大小写字母、数字及常见运算符。

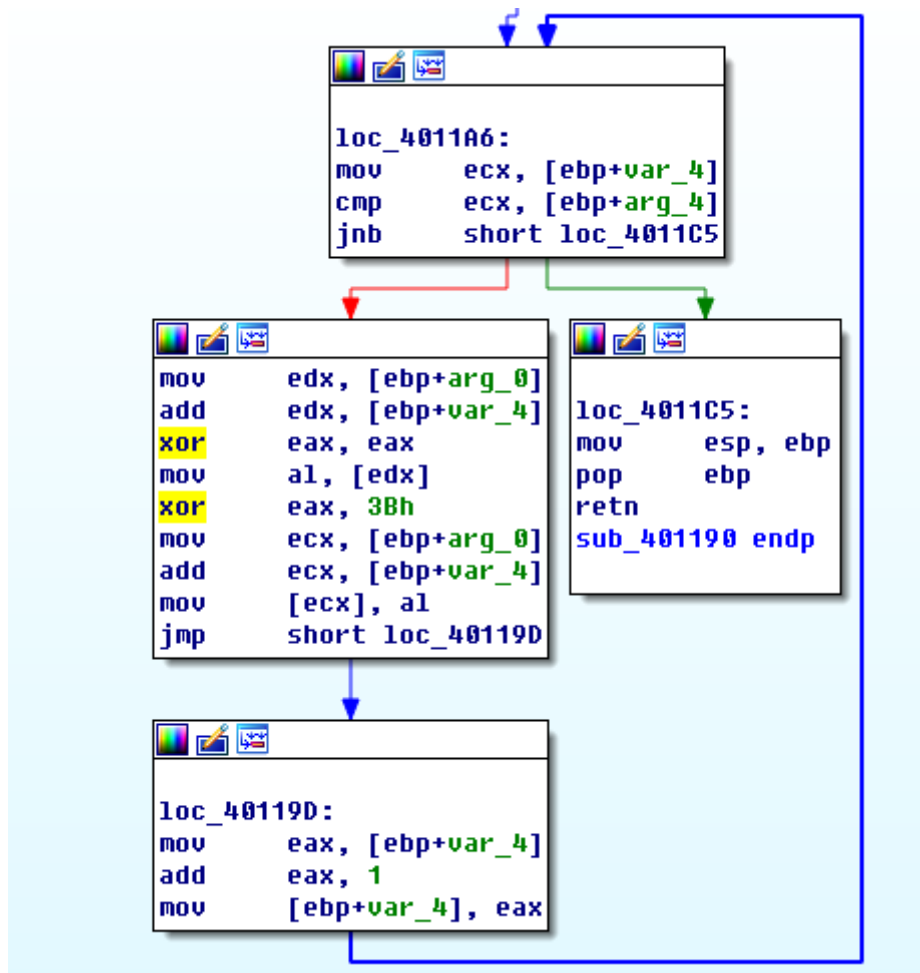
```
_ ^ !
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-
EEE
<8PX
700WP
'h'
ppxxxx
<null>
```

推测部分内容进行了编码加密。

- 使用IDA Pro深入分析

用IDAPro搜索所有非归零xor指令，可以找到3个，但是其中两个(0x00402BE2 和 0x00402BE6 处)是库代码。这也是搜索窗口未列出函数名的原因。可以忽略这些代码，只留下 `xor eax, 3Bh` 指令。

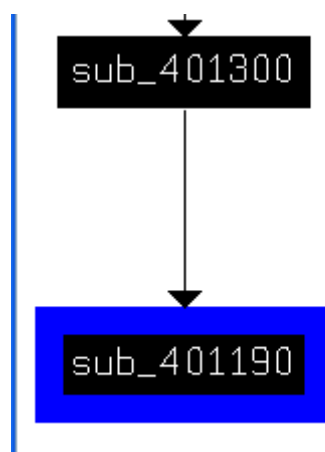
在函数 `sub_401190` 中：



包含一个小的循环，很明显它递增一个计数器(`var_4`)，并且用0x3B与缓冲区(`arg_8`)中原始内容的异或结果来修改缓冲区。其他被异或的参数(`arg_4`)是缓冲区的长度。

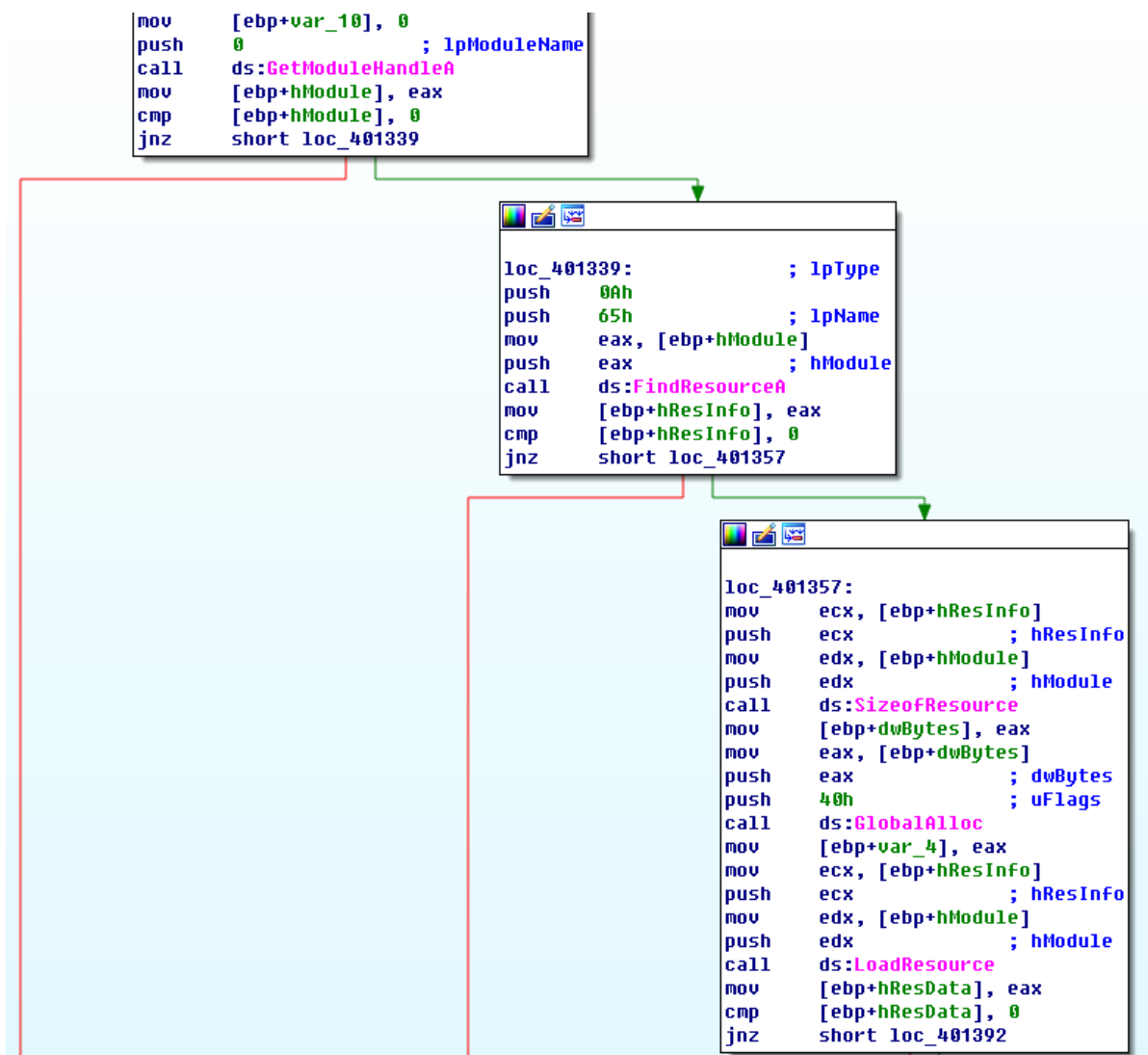
对于函数 `sub_401198`，用静态字节 0x3B 执行单字节XOR加密，并将缓冲区及其长度作为它的参数。

查看 `sub_401198` 引用：



可以看到函数 `sub_401300` 是调用 `sub_401198` 的唯一函数。

跟踪函数 `sub_401300` 调用 `sub_401198` 之前的代码块：



我们依次看到调用 `GetModuleHandleA`、`FindResourceA`、`SizeofResource`、`GlobalAlloc`、`LoadResource`以及 `LockResource`。调用 `sub_401198` 之前，恶意代码会对资源做一些处理。

在这些资源相关的函数中，继续研究找到资源数据的函数 `FindResourceA`：

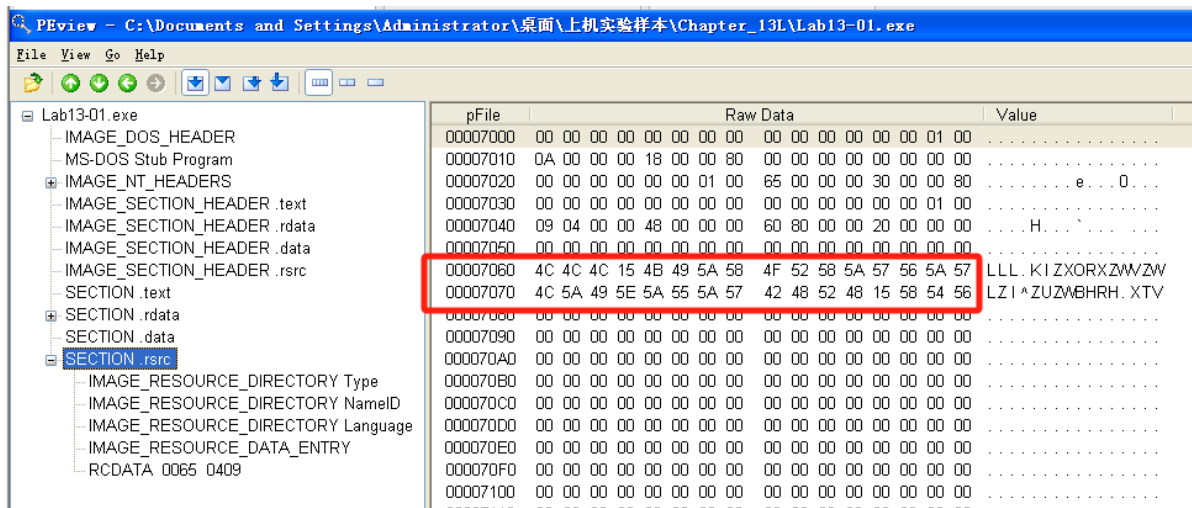
```

loc_401339:                                ; lpType
push    0Ah                               ; lpName
push    65h                               ; hModule
mov     eax, [ebp+hModule]
push    eax                                ; hModule
call    ds:FindResourceA
mov     [ebp+hResInfo], eax
cmp     [ebp+hResInfo], 0
jnz     short loc_401357

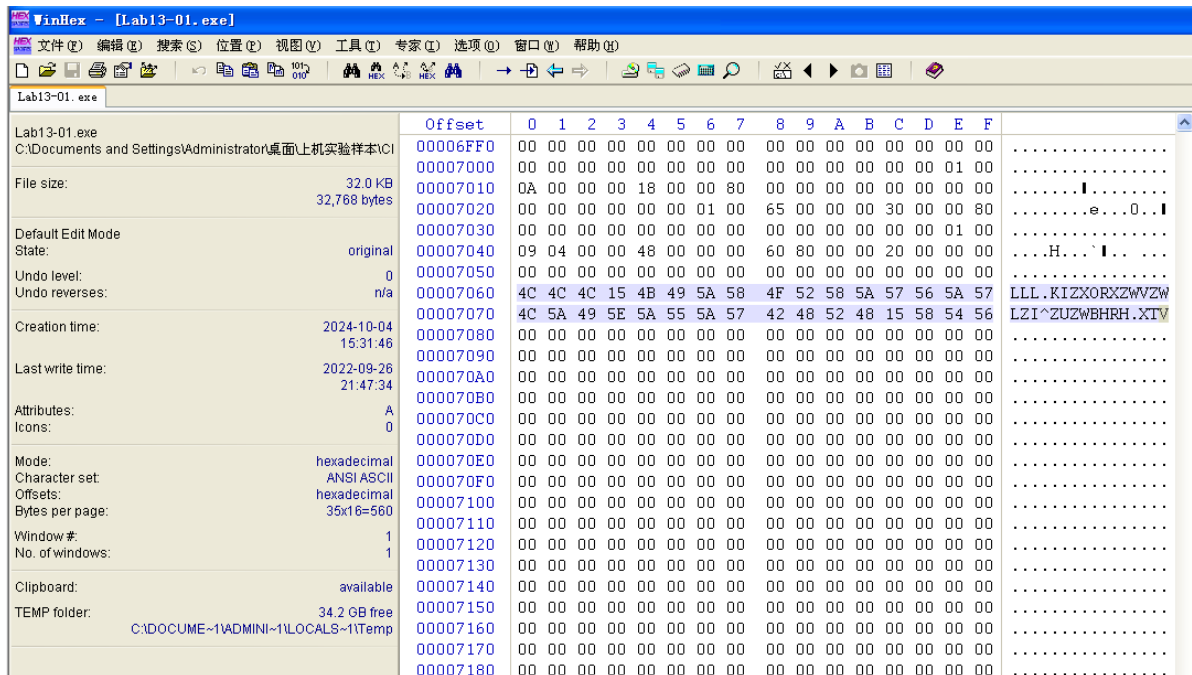
```

IDAPro标注了它的参数。其中 `lpType` 是 `0Ah`，它表示资源数据是应用程序预定义的还是原始数据。`lpName`既可以是一个名字，也可以是一个索引号。本例中，它是一个索引号。

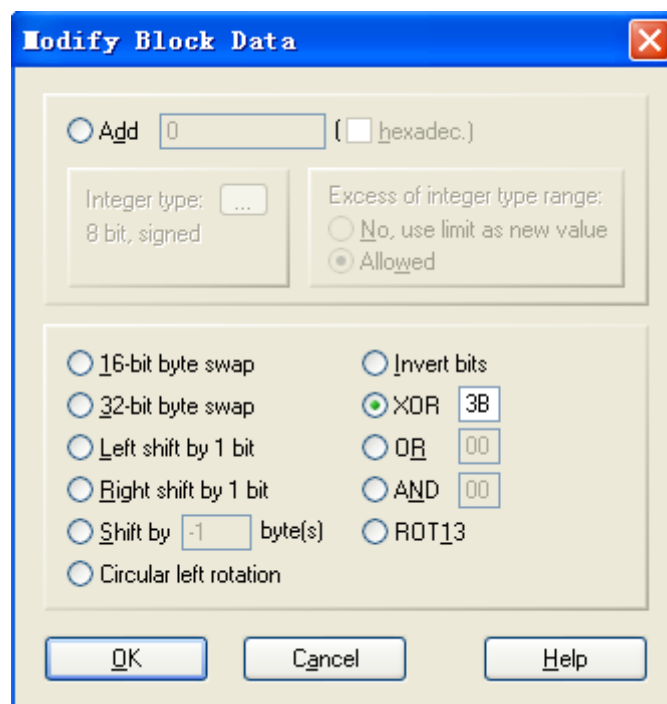
由于函数引用了ID为101的资源，使用PEview查看PE文件中的资源，并且查找索引号为101(0x65)的RCDATA资源，资源在偏移量0x7060处，长度是32字节。



在WinHex中打开该可执行文件：



Modify Data:

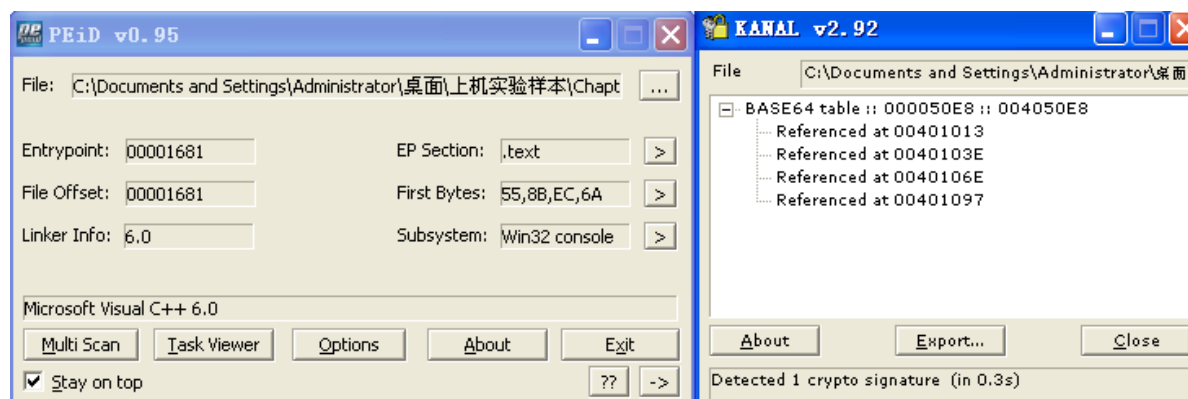


执行结果：

00007050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00007060	77 77 77 2E 70 72 61 63 74 69 63 61 6C 6D 61 6C	www.practicalmal
00007070	77 61 72 65 61 6E 61 6C 79 73 69 73 2E 63 6F 6D	wareanalysis.com
00007080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

显示了用0x3B对每个字节异或之后的结果。上图清晰地显示了以加密形式存储的字符串
www.practicalmalwareanalysis.com。

使用PEiD的KANAL插件，可以看到其在 0x004050E8 处找到一个Base64编码表：

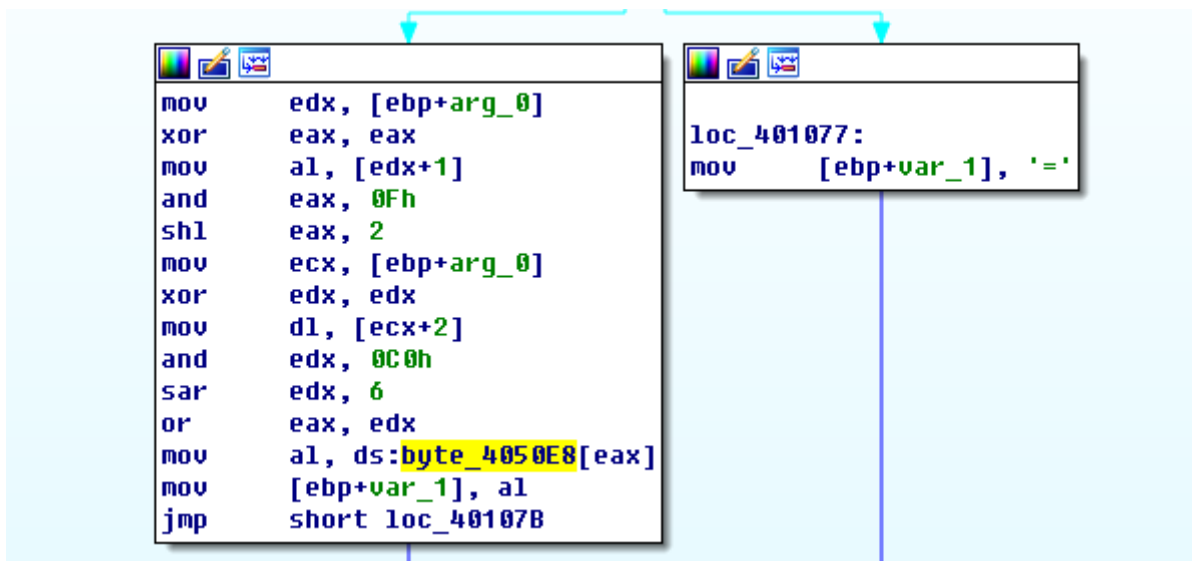
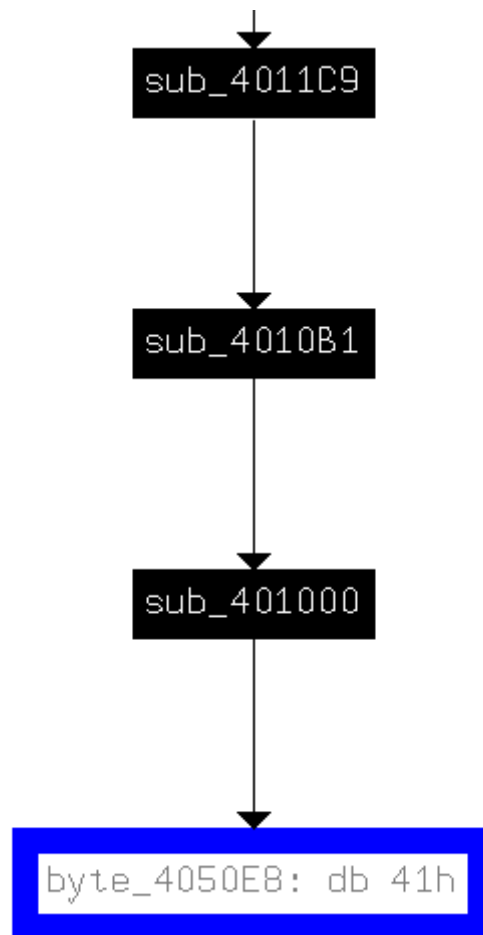


在IDA Pro中跳转到 004050E8：

.rdata:004050E8	
.rdata:004050E9	db 42h ; B
.rdata:004050EA	db 43h ; C
.rdata:004050EB	db 44h ; D
.rdata:004050EC	db 45h ; E
.rdata:004050ED	db 46h ; F
.rdata:004050EE	db 47h ; G
.rdata:004050EF	db 48h ; H
.rdata:004050F0	db 49h ; I
.rdata:004050F1	db 4Ah ; J
.rdata:004050F2	db 4Bh ; K
.rdata:004050F3	db 4Ch ; L
.rdata:004050F4	db 4Dh ; M
.rdata:004050F5	db 4Eh ; N
.rdata:004050F6	db 4Fh ; O
.rdata:004050F7	db 50h ; P
.rdata:004050F8	db 51h ; Q
.rdata:004050F9	db 52h ; R
.rdata:004050FA	db 53h ; S
.rdata:004050FB	db 54h ; T
.rdata:004050FC	db 55h ; U
.rdata:004050FD	db 56h ; V
.rdata:004050FE	db 57h ; W
.rdata:004050FF	db 58h ; X
.rdata:00405100	db 59h ; Y
.rdata:00405101	db 5Ah ; Z
.rdata:00405102	db 61h ; a
.rdata:00405103	db 62h ; b
.rdata:00405104	db 63h ; c
.rdata:00405105	db 64h ; d
.rdata:00405106	db 65h ; e
.rdata:00405107	db 66h ; f

可以看到编码表内容为大小写字母、数字以及几个常见运算符，与前面字符串分析相吻合。

查看这个字符串的引用：



位于以 0x00401000 开始的函数中。右侧的框中有一个字符“=”的交叉引用。这证实：0x00401000 与 Base64 编码相关，因为在 Base64 加密中“=”作为填充字符。

继续查看 sub_004010B1 函数：

```

.text:004011FA      lea     edx, [ebp+name]
.text:00401200      push    edx                ; name
.text:00401201      call   gethostname
.text:00401206      mov     [ebp+var_4], eax
.text:00401209      push    0Ch                ; size_t
.text:0040120B      lea     eax, [ebp+name]
.text:00401211      push    eax                ; char *
.text:00401212      lea     ecx, [ebp+var_18]
.text:00401215      push    ecx                ; char *
.text:00401216      call   _strncpy
.text:0040121B      add     esp, 0Ch
.text:0040121E      mov     [ebp+var_C], 0
.text:00401222      lea     edx, [ebp+var_30]
.text:00401225      push    edx                ; int
.text:00401226      lea     eax, [ebp+var_18]
.text:00401229      push    eax                ; char *
.text:0040122A      call   sub_4010B1
.text:0040122F      add     esp, 8
.text:00401232      mov     byte ptr [ebp+var_23+3], 0
.text:00401236      lea     ecx, [ebp+var_30]
.text:00401239      push    ecx
.text:0040123A      mov     edx, [ebp+arg_0]
.text:0040123D      push    edx
.text:0040123E      push    offset aHttpSS     ; "http://%s/%s/"
.text:00401243      lea     eax, [ebp+szUrl]
.text:00401249      push    eax                ; char *
.text:0040124A      call   _sprintf

```

sub_004010B1 是真正的Base64加密函数。它的目的是将源字符串划分成3个字节的块，并且将3个字节块传递给 sub_00401000，从而将传入的3个字节加密成4个字节。

看一下传递到函数的目的字符串，可以看到，字符串作为 sprintf 的第4个参数入栈。具体说，以 http://%s/%s/ 格式的第2个字符串是URI的路径。

跟踪传递给函数base64encode的源字符串，可以看到，它是strncpy函数的输出。并且strncpy的输入是 gethostname 的输出。因此可以明白加密的URI路径源字符串是主机名。strncpy函数仅复制了主机名的前12个字节。

```

.text:0040125E      call    ds:InternetOpenA
.text:00401264      mov     [ebp+hInternet], eax
.text:0040126A      push    0                  ; dwContext
.text:0040126C      push    0                  ; dwFlags
.text:0040126E      push    0                  ; dwHeadersLength
.text:00401270      push    0                  ; lpszHeaders
.text:00401272      lea     edx, [ebp+szUrl]
.text:00401278      push    edx                ; lpszUrl
.text:00401279      mov     eax, [ebp+hInternet]
.text:0040127F      push    eax                ; hInternet
.text:00401280      call   ds:InternetOpenUrlA
.text:00401286      mov     [ebp+hFile], eax
.text:0040128C      cmp     [ebp+hFile], 0
.text:00401293      jnz     short loc_4012A6
.text:00401295      mov     ecx, [ebp+hInternet]
.text:0040129B      push    ecx                ; hInternet
.text:0040129C      call   ds:InternetCloseHandle
.text:004012A2      xor     al, al
.text:004012A4      jmp     short loc_4012FC
.text:004012A6      ; -----
.text:004012A6      loc_4012A6:                ; CODE XREF: sub_4011C9+CA↑j
.text:004012A6      lea     edx, [ebp+dwNumberOfBytesRead]
.text:004012A9      push    edx                ; lpdwNumberOfBytesRead
.text:004012AA      push    200h               ; dwNumberOfBytesToRead
.text:004012AF      lea     eax, [ebp+Buffer]
.text:004012B5      push    eax                ; lpBuffer
.text:004012B6      mov     ecx, [ebp+hFile]
.text:004012BC      push    ecx                ; hFile
.text:004012BD      call   ds:InternetReadFile
.text:004012C3      mov     [ebp+var_8], eax

```


查看剩余代码，可以看到它使用了WinINet(InternetOpenA、InternetopenUrlA和InternetReadFile)打开并且读取前面代码块中构成的URL。返回数据的第一个字符与字母o比较。如果第一个字母是o，则beacon返回1，否则返回0。主函数由一个循环Sleep调用以及beacon调用组成。当beacon(0x004011C9)返回真时(获得以o开始的Web响应)则循环退出程序终止。

概括来说，这个恶意代码发送通信信号beacon，让攻击者知道它在正常运行。恶意代码用加密的(有可能截断)主机名作为标识符发出一个特定的通信信号，当接收到一个特定的回应后，则终止。

2. 问题解答

(1) 比较恶意代码中的字符串（字符串命令的输出）与动态分析提供的有用信息，基于这些比较，哪些元素可能被加密？

网络中出现两个恶意代码中不存在的字符串(当strings 命令运行时，并没有字符串输出)，一个字符串是域名 www.practicalmalwareanalysis.com，另外一个GET请求路径：aG9zdG5hbWUtZm9v。

(2) 使用 IDA Pro 搜索恶意代码中字符串 'xor'，以此来查找潜在的加密，你发现了哪些加密类型？

004011B8 处的 XOR 指令是 sub_401190 函数中的一个单字节 XOR 加密循环的指令。

(3) 恶意代码使用什么密钥加密，加密了什么内容？

单字节XOR加密使用字节3Bh。用101索引原始的数据源解密的XOR加密缓冲区的内容是www.practicalmalwareanalysis.com。

(4) 使用静态工具 FindCrypt2、Krypto ANALyzer (KANAL) 以及 IDA 熵插件识别一些其他类型的加密机制，你发现了什么？

使用插件 PEiD KANAL，可识别出恶意代码使用标准的 Base64 编码字符串:ABCDEFGHIJKLMNQRSTUvwxyz0123456789+/-

(5) 什么类型的加密被恶意代码用来发送部分网络流量？

标准的 Base64 编码用来创建 GET 请求字符串。

(6) Base64 编码函数在反汇编的何处？

从 0x004010B1 地址处开始。

(7) 恶意代码发送的 Base64 加密数据的最大长度是什么？加密了什么内容？

Base64 加密前，Lab13-01.exe 复制最大 12 个字节的主机名，使得 GET 请求的字符串的最大字符个数是 16，加密的内容为请求的字符串。

(8) 恶意代码中，你是否在 Base64 加密数据中看到了填充字符 (= 或者 ==) ？

如果主机名小于 12 个字节且不能被 3 整除，则可能使用填充字符"="。

(9) 这个恶意代码做了什么？

恶意代码用加密的主机名作为标识符发送一个特定的通信信号，当接收到一个特定的回应后则退出。

Lab13-02

1.过程分析

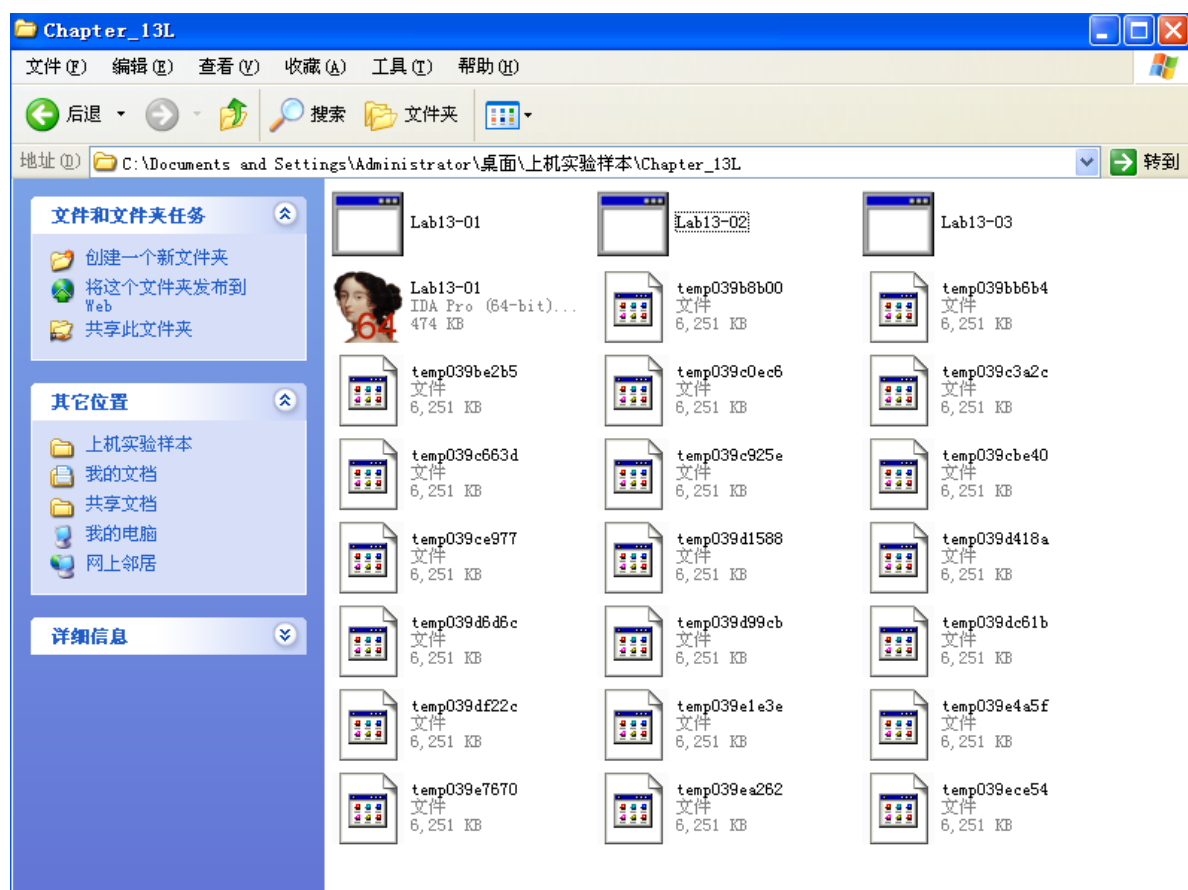
使用strings工具查看该恶意代码的字符串列表

```
LoadLibraryA
SetStdHandle
MultiByteToWideChar
LMapStringA
LMapStringW
GetStringTypeA
GetStringTypeW
FlushFileBuffers
```

可以看到与字节运算相关操作有关的字符串

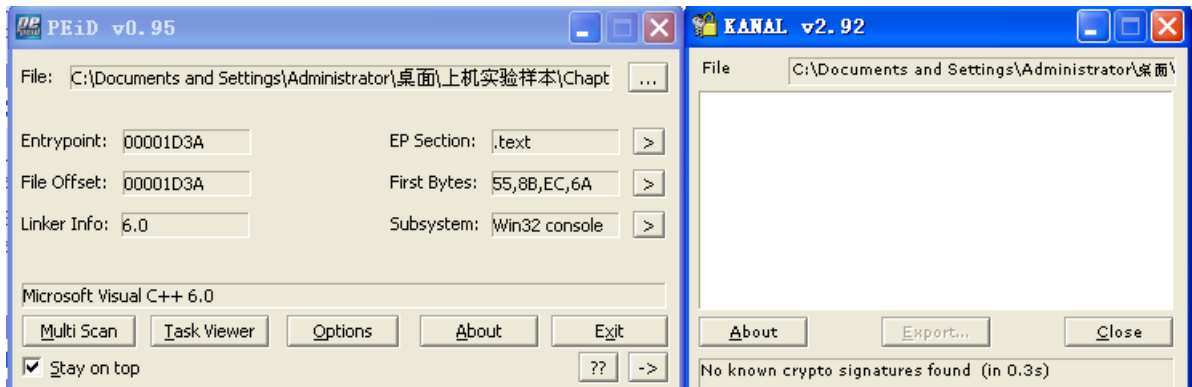
动态分析

运行Lab13-02.exe



启动恶意代码，可以看到它在固定的时间间隔内在恶意代码的当前目录下创建了一些新文件。这些文件相当大(几兆大小)并且文件似乎包含一些随机的数据，文件名以temp开始并且以一些看似随机的字符结束。

继续使用上一恶意代码使用的PEiD中KANAL插件：



发现没有查看到任何有效信息。

使用IDA Pro继续分析：

搜索xor指令，搜索结果：

Address	Function	Instruction
.text:00401040	sub_401000	xor eax, eax
.text:004012D6	sub_40128D	xor eax, [ebp+var_10]
.text:0040171F		xor eax, [esi+edx*4]
.text:0040176F	sub_401739	xor edx, [ecx]
.text:0040177A	sub_401739	xor edx, ecx
.text:00401785	sub_401739	xor edx, ecx
.text:00401795	sub_401739	xor eax, [edx+8]
.text:004017A1	sub_401739	xor eax, edx
.text:004017AC	sub_401739	xor eax, edx
.text:004017BD	sub_401739	xor ecx, [eax+10h]
.text:004017C9	sub_401739	xor ecx, eax
.text:004017D4	sub_401739	xor ecx, eax
.text:004017E5	sub_401739	xor edx, [ecx+18h]
.text:004017F1	sub_401739	xor edx, ecx
.text:004017FC	sub_401739	xor edx, ecx
.text:0040191E	_main	xor eax, eax
.text:0040311A		xor dh, [eax]
.text:0040311E		xor [eax], dh
.text:00403688		xor ecx, ecx
.text:004036A5		xor edx, edx

找到两个有价值的异或指令：分别位于 0040128D 和 00401739。在 00401570 定义函数导致创建包含了前面孤立的 XOR 指令函数。这个未使用的函数也与同组内可能的加密函数有关联。（命名 sub_401739 为 heavy_xor，sub_40128D 为 single_xor）

查看 heavy_xor 函数：

```

var_4= dword ptr -4
arg_0= dword ptr 8
arg_4= dword ptr 0Ch
arg_8= dword ptr 10h
arg_C= dword ptr 14h

push    ebp
mov     ebp, esp
push    ecx
mov     [ebp+var_4], 0
jmp     short loc_40174F

```

```

loc_40174F:
mov     ecx, [ebp+var_4]
cmp     ecx, [ebp+arg_C]
jnb     loc_40181B

```

```

mov     edx, [ebp+arg_0]
push    edx
call    sub_4012DD
add     esp, 4
mov     eax, [ebp+arg_4]
mov     ecx, [ebp+arg_0]
mov     edx, [eax]
xor     edx, [ecx]
mov     eax, [ebp+arg_0]

```

```

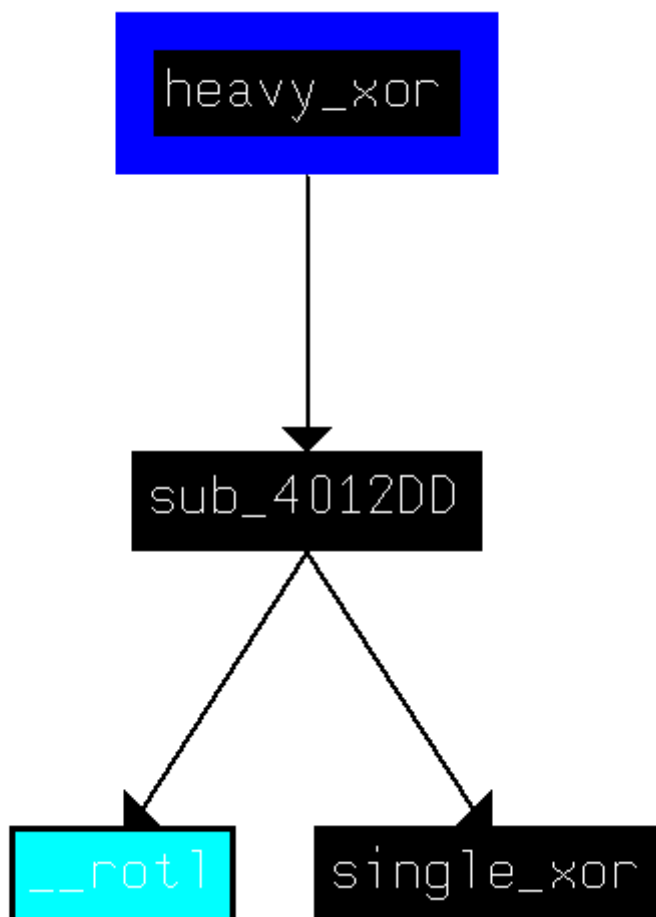
loc_40181B:
mov     esp, ebp
pop     ebp
retn
heavy_xor endp

```

heavy_xor带有4个参数，它是一个单循环，除了xor指令以外，拥有包含多个包含SHL和SHR指令的代码块。

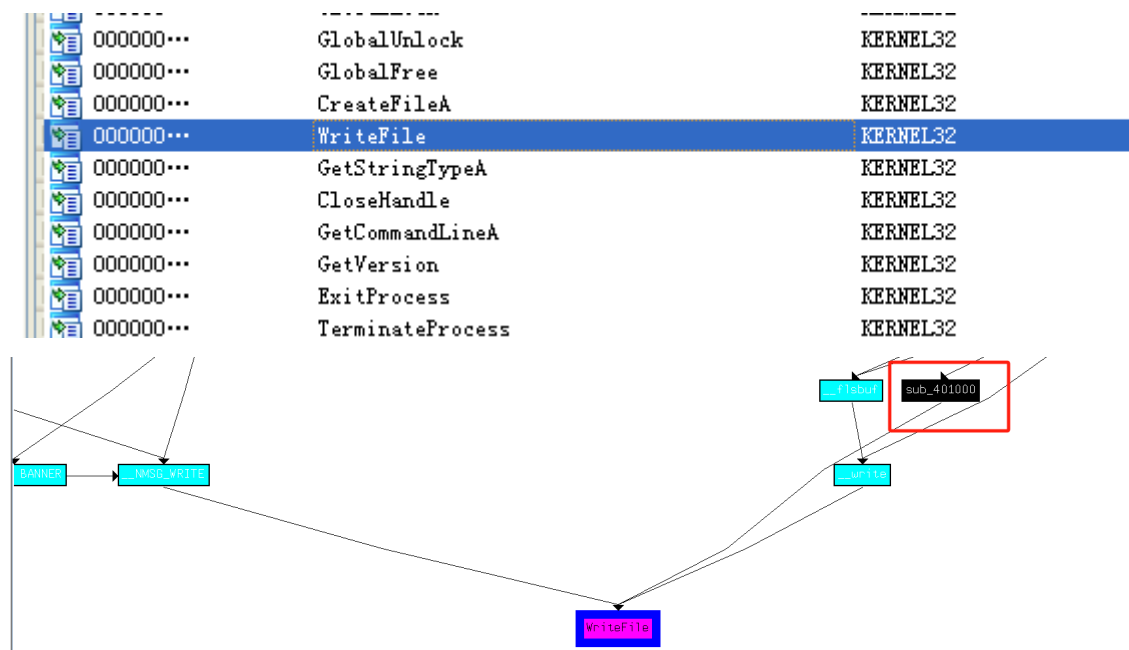
mov	ecx, [ebp+arg_4]	mov	eax, [ebp+arg_0]
shr	ecx, 10h	mov	ecx, [eax+0Ch]
xor	edx, ecx	shl	ecx, 10h
mov	eax, [ebp+arg_0]	xor	edx, ecx
mov	ecx, [eax+0Ch]	mov	eax, [ebp+arg_8]
shl	ecx, 10h	mov	[eax], edx
xor	edx, ecx	mov	ecx, [ebp+arg_4]
mov	eax, [ebp+arg_8]	mov	edx, [ebp+arg_0]
mov	[eax], edx	mov	eax, [ecx+4]
mov	ecx, [ebp+arg_4]	xor	eax, [edx+8]
mov	edx, [ebp+arg_0]	mov	ecx, [ebp+arg_0]
mov	eax, [ecx+4]	mov	edx, [ecx+1Ch]
xor	eax, [edx+8]	shr	edx, 10h
mov	ecx, [ebp+arg_0]	xor	eax, edx
mov	edx, [ecx+1Ch]	mov	ecx, [ebp+arg_0]
shr	edx, 10h	mov	edx, [ecx+14h]
xor	eax, edx	shl	edx, 10h
mov	ecx, [ebp+arg_0]	xor	eax, edx
mov	edx, [ecx+14h]	mov	ecx, [ebp+arg_8]
shl	edx, 10h	mov	[ecx+4], eax
xor	eax, edx	mov	edx, [ebp+arg_4]
mov	ecx, [ebp+arg_8]	mov	eax, [ebp+arg_0]
mov	[ecx+4], eax	mov	ecx, [edx+8]
mov	edx, [ebp+arg_4]	xor	ecx, [eax+10h]
mov	eax, [ebp+arg_0]	mov	edx, [ebp+arg_0]
mov	ecx, [edx+8]	mov	eax, [edx+4]
xor	ecx, [eax+10h]	shr	eax, 10h
mov	edx, [ebp+arg_0]	xor	ecx, eax
mov	eax, [edx+4]	mov	edx, [ebp+arg_0]
shr	eax, 10h	mov	eax, [edx+1Ch]
xor	ecx, eax	shl	eax, 10h
mov	edx, [ebp+arg_0]	xor	ecx, eax
mov	eax, [edx+1Ch]		
shl	eax, 10h		
xor	ecx, eax		

查看heavy_xor调用的函数：



可以发现single_xor与heavy_xor相关，因为single_xor的调用者也被heavy_xor调用。

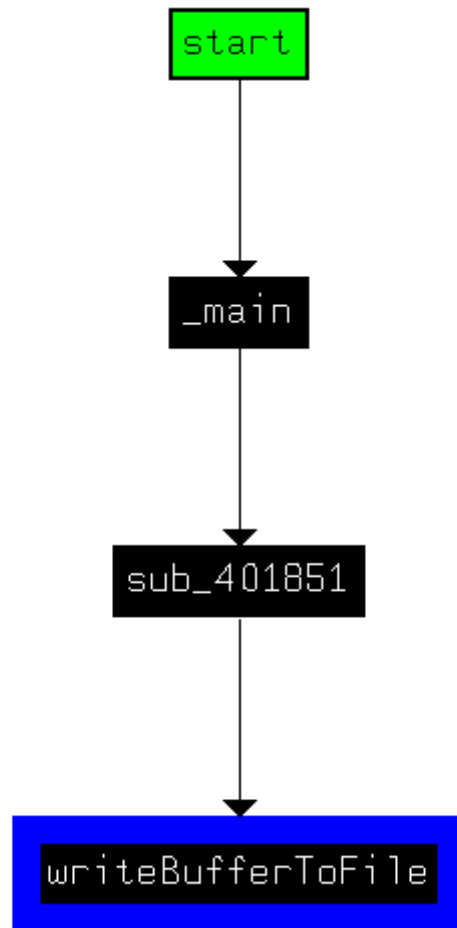
来看一下这个函数与写入到磁盘上temp文件有何种关系。查看一下导入函数看到了WriteFile，检查一下WriteFile的交叉引用。



发现sub_401000，跳转查看：

```
.text:00401000 sub_401000 proc near ; CODE XREF: sub_401851+67↓p
.text:00401000
.text:00401000 hFile = dword ptr -10h
.text:00401000 NumberOfBytesWritten= dword ptr -0Ch
.text:00401000 var_8 = dword ptr -8
.text:00401000 var_4 = dword ptr -4
.text:00401000 lpBuffer = dword ptr 8
.text:00401000 nNumberOfBytesToWrite= dword ptr 0Ch
.text:00401000 lpFileName = dword ptr 10h
.text:00401000
.text:00401000 push ebp
.text:00401001 mov ebp, esp
.text:00401003 sub esp, 10h
.text:00401006 mov [ebp+NumberOfBytesWritten], 0
.text:0040100D mov [ebp+var_4], 0
.text:00401014 mov [ebp+var_8], 0
.text:0040101B push 0 ; hTemplateFile
.text:0040101D push 80h ; dwFlagsAndAttributes
.text:00401022 push 2 ; dwCreationDisposition
.text:00401024 push 0 ; lpSecurityAttributes
.text:00401026 push 0 ; dwShareMode
.text:00401028 push 40000000h ; dwDesiredAccess
.text:0040102D mov eax, [ebp+lpFileName]
.text:00401030 push eax ; lpFileName
.text:00401031 call ds:CreateFileA
.text:00401037 mov [ebp+hFile], eax
.text:0040103A cmp [ebp+hFile], 0FFFFFFFFh
.text:0040103E jnz short loc_401044
.text:00401040 xor eax, eax
```

它用一个缓冲区，一个缓冲区长度，一个文件名作为参数，它打开一个文件并且将缓冲区数据写入到这个文件（重命名sub_401000为writeBufferToFile）。



sub_401851 是调用writeBufferToFile的唯一函数（命名为dostuffAndwriteFile）：

```

    lea     eax, [ebp+nNumberOfBytesToWrite]
    push    eax
    lea     ecx, [ebp+hMem]
    push    ecx
    call    sub_401070
    add     esp, 8
    mov     edx, [ebp+nNumberOfBytesToWrite]
    push    edx
    mov     eax, [ebp+hMem]
    push    eax
    call    sub_40181F
    add     esp, 8
    call    ds:GetTickCount
    mov     [ebp+var_4], eax
    mov     ecx, [ebp+var_4]
    push    ecx
    push    offset aTemp08x ; "temp%08x"
    lea     edx, [ebp+FileName]
    push    edx ; char *
    call    _sprintf
    add     esp, 0Ch
    lea     eax, [ebp+FileName]
    push    eax ; lpFileName
    mov     ecx, [ebp+nNumberOfBytesToWrite]
    push    ecx ; nNumberOfBytesToWrite
    mov     edx, [ebp+hMem]
    push    edx ; lpBuffer
    call    writeBufferToFile
  
```

发现两处函数调用 sub_401070 和 sub_40181F，都是用缓冲区以及缓冲区的长度作为参数。

结合 GetTickCount 结果的格式化字符串"temp%08x"发现了原文件名，它是当前时间的十六进制打印。

sub_40181F 用来加密内容，函数首先设置了4个加密的参数：一个使用之前清空的本地变量，从 dostuffAndwriteFile 传递过来指向同一个缓冲区的两个指针以及一个从 dostuffAndwriteFile 传递过来的缓冲区大小。两个指针指向同一个内存缓冲区暗示编码函数携带源和目的缓冲区以及一个长度，这种情况下，加密将在适当的地方执行。

```
.text:0040181F sub_40181F      proc near                ; CODE XREF: dostuffAndwriteFile+2F↓p
.text:0040181F
.text:0040181F var_44      = byte ptr -44h
.text:0040181F arg_0       = dword ptr  8
.text:0040181F arg_4       = dword ptr  0Ch
.text:0040181F
.text:0040181F      push    ebp
.text:00401820      mov     ebp, esp
.text:00401822      sub     esp, 44h
.text:00401825      push    44h                ; size_t
.text:00401827      push    0                  ; int
.text:00401829      lea     eax, [ebp+var_44]
.text:0040182C      push    eax                ; void *
.text:0040182D      call    _memset
.text:00401832      add     esp, 0Ch
.text:00401835      mov     ecx, [ebp+arg_4]
.text:00401838      push    ecx
.text:00401839      mov     edx, [ebp+arg_0]
.text:0040183C      push    edx
.text:0040183D      mov     eax, [ebp+arg_0]
.text:00401840      push    eax
.text:00401841      lea     ecx, [ebp+var_44]
.text:00401844      push    ecx
.text:00401845      call    heavy_xor
.text:0040184A      add     esp, 10h
.text:0040184D      mov     esp, ebp
.text:0040184F      pop     ebp
.text:00401850      retn
.text:00401850 sub_40181F      endp
```

步入分析第二个函数 heavy_xor，可以看到其调用 XOR 指令，密钥为 10h

```
mov     ecx, [ebp+arg_4]
mov     edx, [eax]
xor     edx, [ecx]
mov     eax, [ebp+arg_0]
mov     ecx, [eax+14h]
shr     ecx, 10h
xor     edx, ecx
mov     eax, [ebp+arg_0]
mov     ecx, [eax+0Ch]
shl     ecx, 10h
xor     edx, ecx
mov     eax, [ebp+arg_8]
mov     [eax], edx
mov     ecx, [ebp+arg_4]
mov     edx, [ebp+arg_0]
mov     eax, [ecx+4]
xor     eax, [edx+8]
mov     ecx, [ebp+arg_0]
mov     edx, [ecx+1Ch]
shr     edx, 10h
xor     eax, edx
mov     ecx, [ebp+arg_0]
mov     edx, [ecx+14h]
shl     edx, 10h
xor     eax, edx
mov     ecx, [ebp+arg_8]
mov     [ecx+4], eax
mov     edx, [ebp+arg_4]
mov     eax, [ebp+arg_0]
mov     ecx, [edx+8]
xor     ecx, [eax+10h]
```


函数 sub_00401070 中可以看到一个拥有多个系统函数的代码块：

```
.text:00401073      sub     esp, 78h
.text:00401076      mov     [ebp+hdc], 0
.text:0040107D      push    0                      ; nIndex
.text:0040107F      call    ds:GetSystemMetrics
.text:00401085      mov     [ebp+var_1C], eax
.text:00401088      push    1                      ; nIndex
.text:0040108A      call    ds:GetSystemMetrics
.text:00401090      mov     [ebp+cy], eax
.text:00401093      call    ds:GetDesktopWindow
.text:00401099      mov     hWnd, eax
.text:0040109E      mov     eax, hWnd
.text:004010A3      push    eax                    ; hWnd
.text:004010A4      call    ds:GetDC
.text:004010AA      mov     hDC, eax
.text:004010AF      mov     ecx, hDC
.text:004010B5      push    ecx                    ; hdc
.text:004010B6      call    ds:CreateCompatibleDC
.text:004010BC      mov     [ebp+hdc], eax
.text:004010BF      mov     edx, [ebp+cy]
.text:004010C2      push    edx                    ; cy
.text:004010C3      mov     eax, [ebp+var_1C]
.text:004010C6      push    eax                    ; cx
.text:004010C7      mov     ecx, hDC
.text:004010CD      push    ecx                    ; hdc
.text:004010CE      call    ds:CreateCompatibleBitmap
.text:004010D4      mov     [ebp+h], eax
.text:004010D7      mov     edx, [ebp+h]
.text:004010DA      push    edx                    ; h
.text:004010DB      mov     eax, [ebp+hdc]
.text:004010DE      push    eax                    ; hdc
.text:004010DF      call    ds:SelectObject
.text:004010E5      push    0CC0020h              ; rop
```

基于这个函数调用的API函数列表，猜测这个函数试图抓屏。

GetDesktopwindow 获取覆盖整个屏幕桌面窗口的一个句柄，函数 BitBlt 和 GetDIBits 获取位图信息并将它们复制到缓冲区。

得出结论：恶意代码反复抓取用户的桌面并且将加密版本的抓屏信息写入到一个文件。

接下来尝试利用抓取到的一个文件，通过加密算法逆向运行它，提取到原始抓屏图片。

创建一个Python脚本，将它以扩展名.py保存在ImmDbg安装目录下的PyScripts文件夹中：

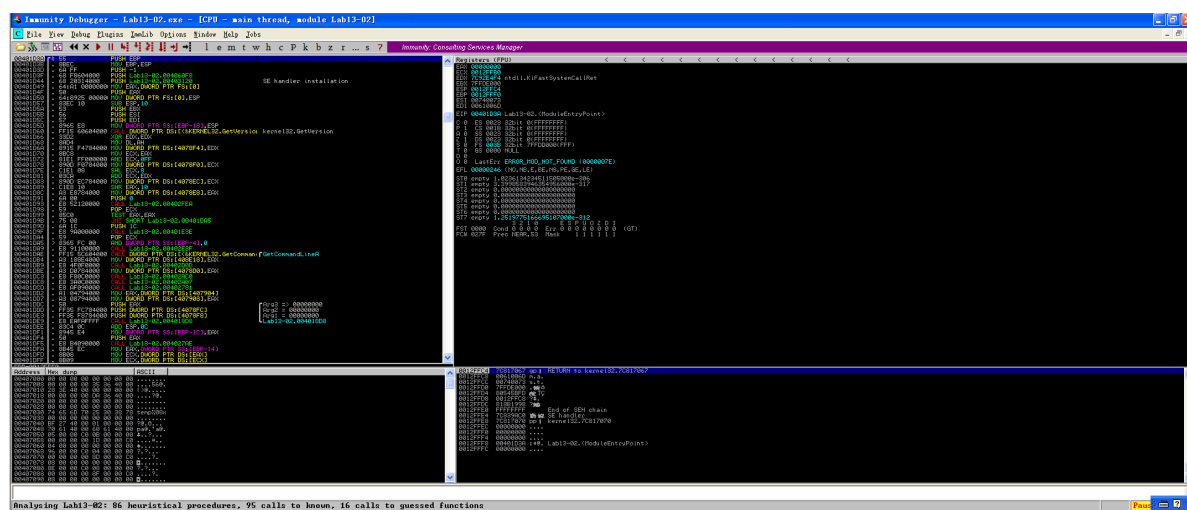
```
#!/usr/bin/env python
import immlib
def main():
    imm = immlib.Debugger()
    imm.setBreakpoint(0x00401875)
    imm.Run()
    cfile = open("c:\\temp062da212", 'rb')
    buffer = cfile.read()
    sz = len(buffer)
    membuf = imm.remoteVirtualAlloc(sz)
    imm.writeMemory(membuf, buffer)
    regs = imm.getRegs()
    imm.writeLong(regs['EBP']-12, membuf)
    imm.writeLong(regs['EBP']-8, sz)
    imm.setBreakpoint(0x0040190A)
    imm.Run()
```

这个Python脚本目的是调试一个目标程序，并在其内存中插入文件数据。

1. 设置断点，运行程序直到到达该断点。
2. 读取一个本地文件的内容并分配足够的内存空间。
3. 将文件内容写入目标程序的内存中。
4. 操作程序的栈，将文件内容和文件大小信息写入到栈的指定位置。
5. 设置另一个断点，继续运行目标程序。

第一个断点刚好在参数被压入堆栈前中断。open调用打开一个已经写入到文件系统的加密文件。随后几行将文件读入内存并且计算缓冲区的大小。remoteVirtualAlloc 调用用来在正在执行的进程内存空间中创建一个大小合适的缓冲区。另外，writeMemory 用来将文件的内容复制到这个新创建的缓冲区中。两个writeLong 调用替换堆栈中加密的缓冲区和它大小的变量。随后几条指令则是将这些变量压入堆栈，供随后的加密例程和写文件使用。

在ImmDbg中打开恶意代码



选择mmLib→RunPyihon Script，然后选择创建的脚本，则脚本将会执行且调试器会在第二个断点处中断。

此时，恶意代码已经在它的当前目录中写了一个文件。进入恶意代码所在目录并且确定它最近写入的文件，将文件的扩展名修改为.bmp并且打开它。可以看到解密后的截图是恶意代码之前的抓屏。

2. 问题解答

(1) 使用动态分析，确定恶意代码创建了什么？

Lab13-02.exe 在当前目录下以固定的时间间隔创建一些新的文件，一些较大且看似随机的文件，这些文件的命名以 temp 开始，以不同的8个十六进制数字结束。

(2) 使用静态分析技术，例如 xor 指令搜索、FidCrypt2、KANAL 以及 IDA 熵插件，查找潜在的加密，你发现了什么？

XOR 搜索技术在 sub_401570 和 sub_401739 中识别了加密相关的函数，其余没有发现

(3) 基于问题 1 的回答，哪些导入函数将是寻找加密函数比较好的一个证据？

WriteFile 函数调用之前可能会发现加密函数。

(4) 加密函数在反汇编的何处？

sub_40181F。

(5) 从加密函数追溯原始的加密内容，原始加密内容是什么？

屏幕截图。

(6) 你是否能够找到加密算法？如果没有，你如何解密这些内容？

加密算法是不标准算法，不容易识别；通过解密工具解密流量。

(7) 使用解密工具，你是否能够恢复加密文件中的一个文件到原始文件？

具体过程见上述分析，恢复思路为：

由于代码已经有指令获取缓冲区，加密它，并写入到文件，因此，以如下方式重新使用它：

1. 执行加密前，让程序正常运行。
2. 使用希望解密的已保存到文件的缓冲区覆盖存放抓屏信息的缓冲区。
3. 让程序将输出结果写入到一个临时文件，它的文件名基于当前时间。
4. 写入第一个文件后，中断程序的运行。

Lab13-03

1.过程分析

使用strings工具查看字符串列表

```
CDEFGHIJKLMNOPQRSTUVWXYZABcdefghijklmnopqrstuvwxyzab0123456789+/  
ERROR: API      = %s.  
      error code = %d.  
      message   = %s.  
ReadFile  
WriteConsole  
ReadConsole  
WriteFile  
dir  
DuplicateHandle  
DuplicateHandle  
DuplicateHandle  
CloseHandle  
CloseHandle  
GetStdHandle
```

可以发现格式字符串以及与 Base64 相关的字符串，推测该恶意代码有加密内容。

同时观察到几个函数：WriteConsole（从当前光标位置开始，将字符串写入控制台屏幕缓冲区）、ReadConsole（从控制台输入缓冲区读取字符输入，并将其从缓冲区中删除）和 DuplicateHandle（复制对象句柄）。

```
ijklmnopqrstuvwxyz  
www.practicalmalwareanalysis.com  
L"A  
d"A
```

同时观察到网页地址 www.practicalmalwareanalysis.com

检查一下xor指令并且去掉与寄存器清零和库函数相关的xor指令，发现6个包含xor指令的函数。

Address	Function	Instruction
.text:004027C6	sub_40223A	xor edx, eax
.text:004027DE	sub_40223A	xor eax, [ebp+var_1C]
.text:004027F9	sub_4027ED	xor ecx, ecx
.text:0040282B	sub_4027ED	xor edx, edx
.text:00402841	sub_4027ED	xor edx, edx
.text:0040285C	sub_4027ED	xor eax, eax
.text:00402877	sub_4027ED	xor ecx, ecx
.text:00402892	sub_4027ED	xor edx, [ecx]
.text:0040289A	sub_4027ED	xor ecx, ecx
.text:004028B0	sub_4027ED	xor ecx, ecx
.text:004028CB	sub_4027ED	xor edx, edx
.text:004028E6	sub_4027ED	xor eax, eax
.text:00402901	sub_4027ED	xor ecx, [eax+4]
.text:0040290A	sub_4027ED	xor eax, eax
.text:00402920	sub_4027ED	xor eax, eax
.text:0040293B	sub_4027ED	xor ecx, ecx
.text:00402956	sub_4027ED	xor edx, edx
.text:00402971	sub_4027ED	xor eax, [edx+8]
.text:0040297A	sub_4027ED	xor edx, edx
.text:00402990	sub_4027ED	xor edx, edx
.text:004029AB	sub_4027ED	xor eax, eax
.text:004029C6	sub_4027ED	xor ecx, ecx
.text:004029E1	sub_4027ED	xor edx, [ecx+0Ch]
.text:00402A3C	sub_4027ED	xor edx, ds:word_40DF08[ecx*4]
.text:00402A4E	sub_4027ED	xor edx, ds:word_40E308[ecx*4]
.text:00402A5E	sub_4027ED	xor edx, ds:word_40E708[ecx*4]
.text:00402A68	sub_4027ED	xor edx, [eax]
.text:00402A8C	sub_4027ED	xor eax, ds:word_40DF08[edx*4]
.text:00402A9F	sub_4027ED	xor eax, ds:word_40E308[ecx*4]
.text:00402AAF	sub_4027ED	xor eax, ds:word_40E708[edx*4]
.text:00402AB9	sub_4027ED	xor eax, [ecx+4]
.text:00402ADD	sub_4027ED	xor ecx, ds:word_40DF08[ecx*4]
.text:00402AF0	sub_4027ED	xor ecx, ds:word_40E308[edx*4]

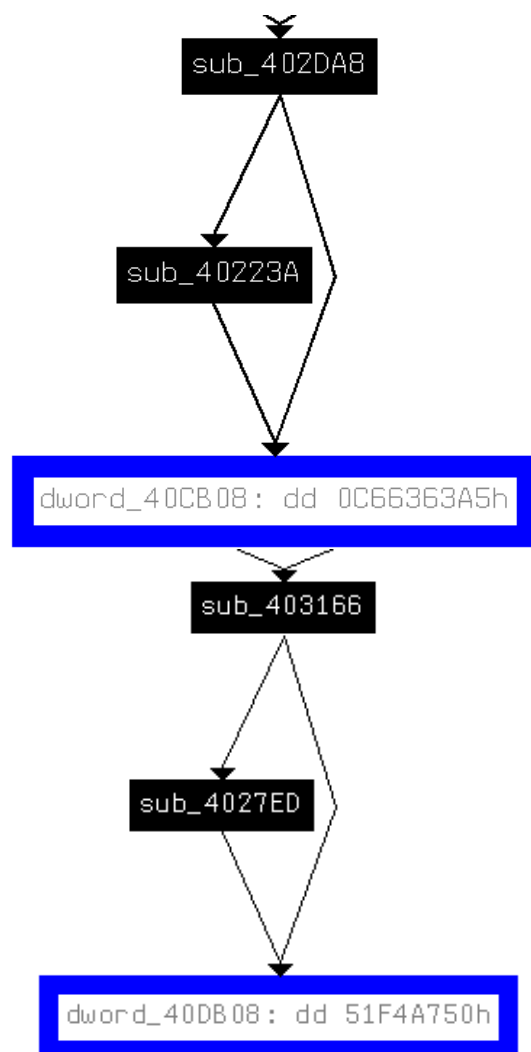
用IDA Pro的FindCrypt2插件，可以发现如下图所示的常量，共8个：

```

40CB08: found const array Rijndael_Te0 (used in Rijndael)
40CF08: found const array Rijndael_Te1 (used in Rijndael)
40D308: found const array Rijndael_Te2 (used in Rijndael)
40D708: found const array Rijndael_Te3 (used in Rijndael)
40DB08: found const array Rijndael_Td0 (used in Rijndael)
40DF08: found const array Rijndael_Td1 (used in Rijndael)
40E308: found const array Rijndael_Td2 (used in Rijndael)
40E708: found const array Rijndael_Td3 (used in Rijndael)
Found 8 known constant arrays in total.

```

可以看到是Rijndael算法，它是AES加密的曾用名。



交叉引用后，看到0040223A和00402DA8通过加密常量(TeX)关联，并且004027ED和00403166通过解密常量(Tdx)关联。（这些函数都是前面发现的包含xor指令的函数）



PEiD KANAL插件在一个相似位置显示了这个AES常量。PEiD识别的S和S-inv参考了S-box结构，它是一些加密算法的基本结构。

IDA 熵插件显示了高熵的区域。检查高8比特位的区域(256位的块大小的最小熵值是7.9)高亮显示了0x0040C900到0x0040CB00之间的区域——与前面识别的S-box区域相同。

查看到以0x004120A4开始的一个字符串包含所有Base64编码的字符，与先前静态分析中查看到的字符串一致：

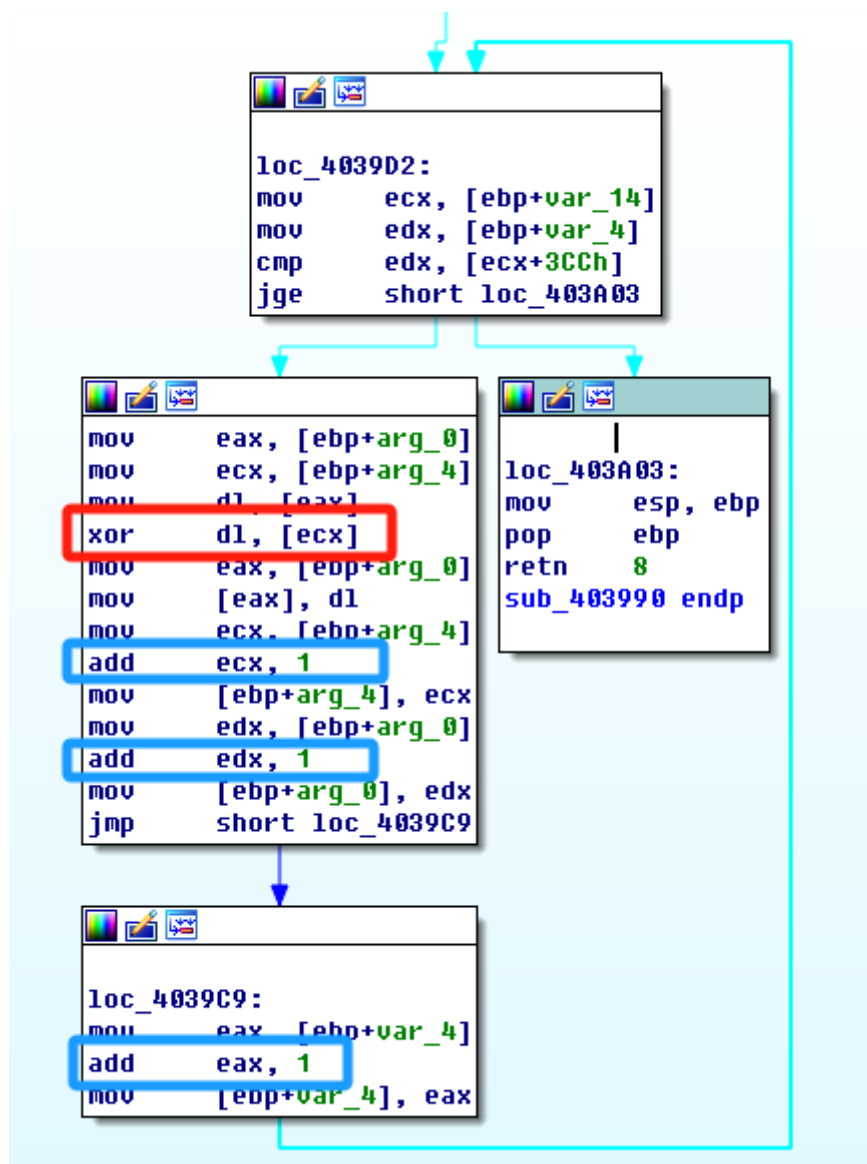
```

.data:004120A4 byte_4120A4 db 43h ; DATA XREF: sub_40103F+1F↑r
.data:004120A5 aDefghi jklmnopq db 'DEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/',0

```

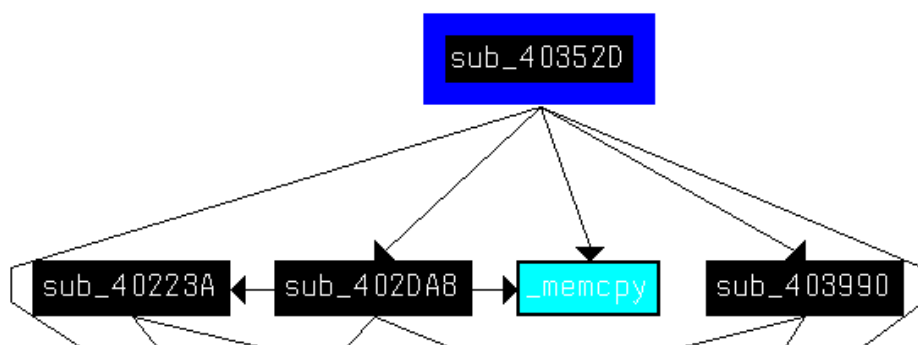
注意到这不是标准的Base64编码字符串，因为大写字母AB和小写字母ab已经被转移到后面的大写或者小写段的后面。这个恶意代码使用了自定义的Base64加密算法。

查看函数00403990:



红框标注的xor指令显示该函数被用来进行XOR加密。变量arg0是个指针，指向用来进行转换的原缓冲区，变量arg_4指向用来提供异或原数据的缓冲区。之后的循环，指向两个缓冲区的指针(arg0和arg_4)以及计数器变量var_4，由后续的3个add指令引用更新。

为了判断该函数是否与其他加密函数之间有关联，检查其的交叉引用：



从这个图中，可以看出该函数确实与其他AES加密的函数0040223A和00402DA8相关。

接着重点分析第一个函数。其处理与加密相关信息的初始化并进行判定，检测密钥的异常。

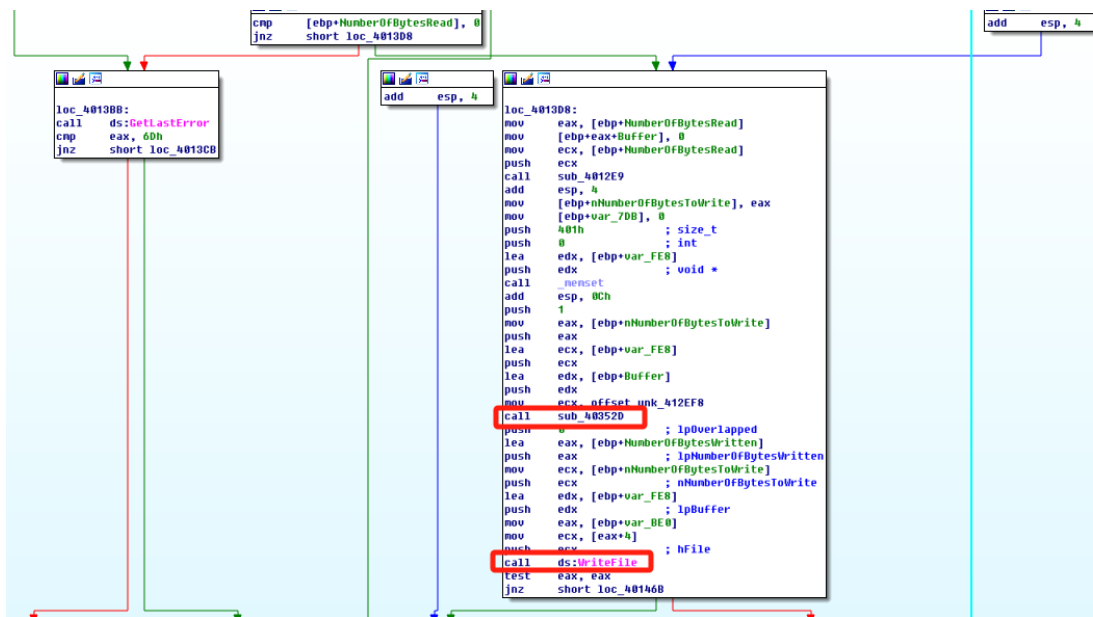
```

mov     [ebp+var_3C], offset aEmptyKey ; "Empty key"
lea     eax, [ebp+var_3C]
push    eax
lea     ecx, [ebp+var_38]
call    ??0exception@QAE@ABQBDQZ ; exception::exception(char const * const &)
push    offset unk_410858
lea     ecx, [ebp+var_38]
push    ecx
call    __CxxThrowException@8 ; _CxxThrowException(x,x)

```

查看其调用情况，可以看到，其调用前 sub_412EF8 首先被调用，将偏移量作为参数传给 s_xor1，推测是一个 AES 加密器；s_xor1 中对 main 中参数进行设定，用于加密。

下面分析 AES 加密算法。0040132B 处调用加密函数，s_xor1 只被调用一次设置密钥。Base64 编码表参与加密，在 0040103F 被调用。函数索引编码表将解密后的字符串分成 32bits 的块，定义一个解码函数，在读文件和写文件之间调用。



查看 main 函数：

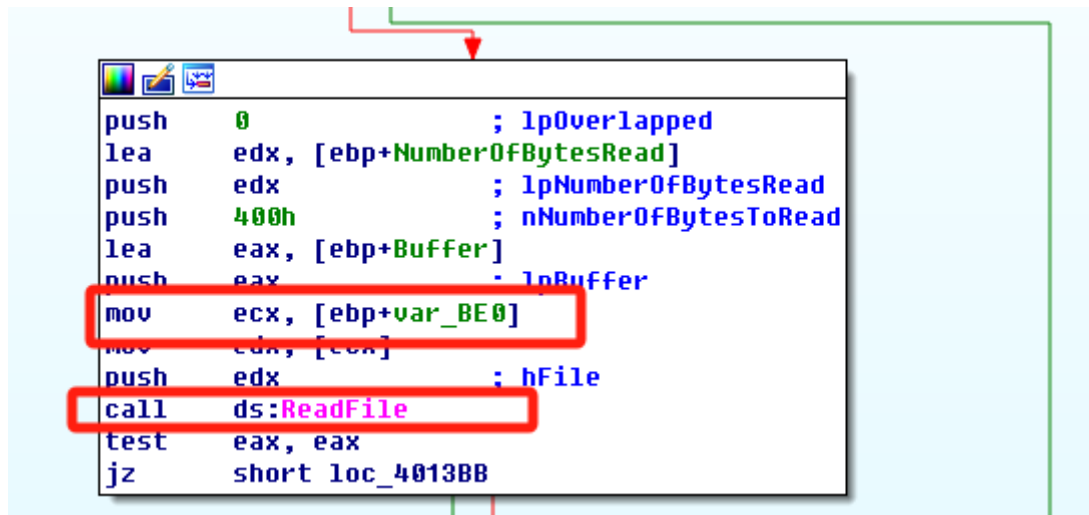
```

loc_401823:
mov     eax, [ebp+var_18]
mov     [ebp+var_58], eax
mov     ecx, [ebp+arg_10]
mov     [ebp+var_54], ecx
mov     edx, dword_41336C
mov     [ebp+var_50], edx
lea     eax, [ebp+var_3C]
push    eax ; lpThreadId
push    0 ; dwCreationFlags
lea     ecx, [ebp+var_58]
push    ecx ; lpParameter
push    offset sub_40132B ; lpStartAddress
push    0 ; dwStackSize
push    0 ; lpThreadAttributes
call    ds:CreateThread
mov     [ebp+var_20], eax
cmp     [ebp+var_20], 0
jnz     short loc_401867

```

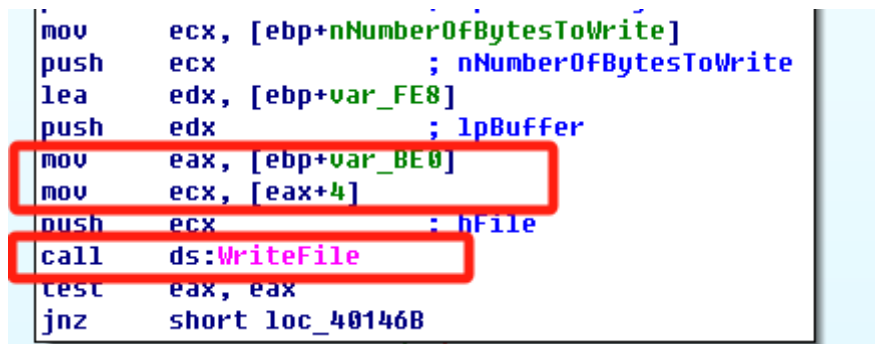
这里创建了一个线程，并且线程的起始地址就是加密函数的起始地址

进入到这个函数里，查看一下参数都是在哪里进行了使用：



```
push    0                ; lpOverlapped
lea     edx, [ebp+NumberOfBytesRead]
push    edx              ; lpNumberOfBytesRead
push    400h             ; nNumberOfBytesToRead
lea     eax, [ebp+Buffer]
push    eax              ; lpBuffer
mov     ecx, [ebp+var_BE0]
mov     edx, [ecx]
push    edx              ; hFile
call    ds:ReadFile
test    eax, eax
jz      short loc_4013BB
```

线程内还有一个writefile的函数，这函数的参数



```
mov     ecx, [ebp+nNumberOfBytesToWrite]
push    ecx              ; nNumberOfBytesToWrite
lea     edx, [ebp+var_FE8]
push    edx              ; lpBuffer
mov     eax, [ebp+var_BE0]
mov     ecx, [eax+4]
push    ecx              ; hFile
call    ds:WriteFile
test    eax, eax
jnz     short loc_40146B
```

参数是arg_10

```
.text:004015B7 var_18          = dword ptr -18h
.text:004015B7 var_14          = dword ptr -14h
.text:004015B7 ProcessInformation = _PROCESS_INFORMATION ptr -10h
.text:004015B7 arg_10          = dword ptr 18h
.text:004015B7
```

往上走可以发现这里其实就是这个函数的一个参数

```
loc_40196E:                                ; CODE XREF: _main+EC↑j
mov     ecx, [ebp+5]
push    ecx
sub     esp, 10h
mov     edx, esp
mov     eax, dword ptr [ebp+name.sa_family]
mov     [edx], eax
mov     ecx, dword ptr [ebp+name.sa_data+2]
mov     [edx+4], ecx
mov     eax, dword ptr [ebp+name.sa_data+6]
mov     [edx+8], eax
mov     ecx, dword ptr [ebp+name.sa_data+0Ah]
mov     [edx+0Ch], ecx
call    sub_4015B7
add     esp, 14h
xor     eax, eax
```

回到函数内，可以发现：


```

loc_401656:                                ; CODE XREF: sub_4015B7+90↑j
        push    2                          ; dwOptions
        push    0                          ; bInheritHandle
        push    0                          ; dwDesiredAccess
        lea     ecx, [ebp+var_18]
        push    ecx                        ; lpTargetHandle
        call    ds:GetCurrentProcess
        push    eax                        ; hTargetProcessHandle
        mov     edx, [ebp+hReadPipe]
        push    edx                        ; hSourceHandle
        call    ds:GetCurrentProcess
        push    eax                        ; hSourceProcessHandle
        call    ds:DuplicateHandle
        test    eax, eax
        jnz     short loc_401689
        push    offset aDuplicatehan_0 ; "DuplicateHandle"
        call    sub_401256

.text:00401770        push    0                          ; lpThreadAttributes
.text:00401772        push    0                          ; lpProcessAttributes
.text:00401774        push    offset CommandLine ; "cmd.exe"
.text:00401779        push    0                          ; lpApplicationName
        call    ds:CreateProcessA
        mov     [ebp+var_34], eax
        mov     eax, [ebp+ProcessInformation.hProcess]
        mov     dword_41336C, eax
        mov     ecx, [ebp+hWritePipe]

```

这一系列的操作就是典型的创建了一个反向shell，建立后门，使用 CreatePorcessA 进行启动。

而根据之前的调用base64和AES的位置可以发现，这两个都是在readfile和writefile之间被调用的，然后base64的调用是在AES之前，也就是说，这两个应该是有一个先后顺序：首先使用base64对传递来的指令进行一个解密操作。然后在本地执行完指令之后，使用AES对执行结果进行加密，并反馈给远端。

具体的解密算法参考指导书中的代码即可对其加密的内容进行解密：

Base64解密脚本：

```

import string
import base64

s = ""
tab = "CDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
b64 = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'
ciphertext = 'BInaEi=='

for ch in ciphertext:
    if ch in tab:
        s += b64[string.find(tab, ch)]
    elif ch == '=':
        s += '='

print(base64.decodestring(s))

```

运行脚本结果为：



AES解密脚本：

```

from Crypto.Cipher import AES

```

```
import binascii

raw = '37 f3 1f 04 51 20 e0 b5 86 ac b6 0f 65 20 89 92 ' + \
      '4f af 98 a4 c8 76 98 a6 4d d5 51 8f a5 cb 51 c5 ' + \
      'cf 86 11 0d c5 35 38 5c 9c c5 ab 66 78 40 1d df ' + \
      '4a 53 f0 11 0f 57 6d 4f b7 c9 c8 bf 29 79 2f c1 ' + \
      'ec 60 b2 23 00 7b 28 fa 4d c1 7b 81 93 bb ca 9e ' + \
      'bb 27 dd 47 b6 be 0b 0f 66 10 95 17 9e d7 c4 8d ' + \
      'ee 11 09 99 20 49 3b df de be 6e ef 6a 12 db bd ' + \
      'a6 76 b0 22 13 ee a9 38 2d 2f 56 06 78 cb 2f 91 ' + \
      'af 64 af a6 d1 43 f1 f5 47 f6 c2 c8 6f 00 49 39'

ciphertext = binascii.unhexlify(raw.replace(' ', ''))
obj = AES.new('ijklmnopqrstuvwxyz', AES.MODE_CBC)
print('Plaintext is:\n' + obj.decrypt(ciphertext).decode('utf-8'))
```

运行脚本结果：

```
Plaintext is:
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.
```

2. 问题解答

(1) 比较恶意代码的输出字符串和动态分析提供的信息，通过这些比较，你发现哪些元素可能被加密？

可以发现格式字符串以及与 Base64 相关的字符串，推测恶意代码使用了加密，但是程序的输出中没有可以识别的字符串，所以没有发现哪些元素可能被加密。

(2) 使用静态分析搜索字符串 xor 来查找潜在的加密。通过这种方法，你发现什么类型的加密？

搜索 xor 指令发现了 6 个可能与加密相关的单独函数，但是加密的类型不确定。

(3) 使用静态工具，如 FindCrypt2、KANAL 以及 IDA 熵插件识别一些其他类型的加密机制。发现的结果与搜索字符 XOR 结果比较如何？

这三种技术都识别了高级加密标准 AES 算法（Rijndael 算法），它与识别的 6 个 XOR 函数相关，IDA 熵插件也能识别一个自定义的 Base64 索引字符串，这表明没有明显的证据与 XOR 指令相关。

(4) 恶意代码使用哪两种加密技术？

恶意代码使用 AES 和自定义的 Base64 加密技术。

(5) 对于每一种加密技术，它们的密钥是什么？

AES 的密钥是：ijklmnopqrstuvwxyz 自定义加密的索引字符串是：

CDEFGHIJKLMNOPQRSTUVWXYZABcdefghijklmnopqrstuvwxyzab0123456789+/

(6) 对于加密算法，它的密钥足够可靠吗？另外你必须知道什么？

对于自定义 Base64 加密的实现，索引字符串已经足够了，但是对于 AES，实现解密可能需要密钥之外的变量，如果使用密钥生成算法，则包括密钥生成算法、密钥大小、操作模式，如果需要还包括向量的初始化等。

(7) 恶意代码做了什么？

恶意代码使用以自定义 Base64 加密算法加密传入命令和以 AES 加密传出 shell 命令响应来建立反连命令 shel，建立后门。

(8) 构造代码来解密动态分析过程中生成的一些内容，解密后的内容是什么？

见上面过程分析。

使用Base64解密得到“dir”，使用AES解密得到一些简单的命令提示。

三、yara规则

基于以上分析编写yara规则如下：

```
rule lab1301exe{
strings:
    $string1 = "Mozilla/4.0"
    $string2 = "http://%s/%s/"
    $string3 = "closeHandle"
    $string4 =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}
rule lab1302exe{
strings:
    $string1 = "56@"
    $string2 = "temp%08x"
    $string3 = "MultiByteToWideChar"
condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}
rule lab1303exe{
strings:
    $string1 =
"CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZab0123456789+/"
    $string2 = "ijklmnopqrstuvwxyz"
    $string3 = "www.practicalmalwareanalysis.com"
condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}
```

运行结果如下，可见所有恶意文件都已被检出：

```
PS D:\大三上\大三上\恶意代码分析与防治技术\Practical Malware Analysis Labs\Practical Malware Analysis Labs\BinaryCollect
ion\Chapter_13L> .\yara64.exe -r -c .\Lab13.yar .
.\test.py: 0
.\Lab13.yar: 0
.\Lab13-02.exe: 1
.\Lab13-03.exe: 1
.\Lab13-01.exe: 1
```

四、IDA Python脚本编写

遍历所有函数，排除库函数或简单跳转函数，当反汇编的助记符为call或者jmp且操作数为寄存器类型时，输出该行反汇编指令。

定位当前函数：

- 获取光标所在函数的名称、起止地址。

检查函数属性：

- 打印函数是否具有特定标志位（如 `FUNC_NORET`、`FUNC_FRAME` 等）。

分析函数内部指令：

- 如果函数不是库函数或跳板函数，则列出函数内的所有 `call` 和 `jmp` 指令的地址和反汇编内容。

```
import idutils
ea=idc.ScreenEA()
funcName=idc.GetFunctionName(ea)
func=idaapi.get_func(ea)
print("FuncName:%s"%funcName)
print "Start:0x%x,End:0x%x" % (func.startEA,func.endEA)

flags = idc.GetFunctionFlags(ea)
if flags&FUNC_NORET:
    print "FUNC_NORET"
if flags & FUNC_FAR:
    print "FUNC_FAR"
if flags & FUNC_STATIC:
    print "FUNC_STATIC"
if flags & FUNC_FRAME:
    print "FUNC_FRAME"
if flags & FUNC_USERFAR:
    print "FUNC_USERFAR"
if flags & FUNC_HIDDEN:
    print "FUNC_HIDDEN"
if flags & FUNC_THUNK:
    print "FUNC_THUNK"
if not(flags & FUNC_LIB or flags & FUNC_THUNK):# 获取当前函数中call或者jmp的指令
    dism_addr = list(idutils.FuncItems(ea))
    for line in dism_addr:
        m = idc.GetMnem(line)
        if m == "call" or m == "jmp":
            print "0x%x %s" % (line,idc.GetDisasm(line))
```

在Lab11-03.dll的IDA Pro中运行该脚本

分析出光标所在位置的函数main的call指令和jmp指令：

```

FuncName: _main
Start: 0x401879, End: 0x4019ab
FUNC_FRAME
0x401895 call    sub_401AC2
0x4018a6 call    ds:WSAStartup
0x4018c0 jmp     loc_4019A7
0x4018d1 call    ds:WSASocketA
0x4018eb jmp     loc_4019A7
0x4018f5 call    ds:gethostbyname
0x40190f jmp     loc_4019A7
0x40192c call    ds:htons
0x401952 call    ds:connect
0x40196c jmp     short loc_4019A7
0x40199d call    sub_4015B7

```

五、实验结论及心得体会

1. 本次实验中各恶意代码都使用了不同的加密方法：

Lab13-01中使用了单字节XOR加密，以及base64编码，解码方式较简单，只要找到加密所用的密钥即可使用工具快速解密；

Lab13-02中使用的是非标准加密算法，可以找到加密相关的函数，需使用解密工具解密流量，并恢复原文件；

Lab13-03中使用了AES 和自定义的 Base64 加密技术，是两种标准加密算法，可以根据找到的字符串密钥进行脚本解密。

2. 在Lab13-03中，直接提供了密钥。给定解密方案可能自己拥有生成假定用户提供的密钥或者基于字符串的密码的技术。在这种情况下，需要识别密钥生成算法并且分开备份。在一个标准算法中，可能必须正确地指定选项。例如，一个单独的加密算法可能允许多个密钥大小、块大小、加密和解密轮数以及填充策略。
3. 同时在Lab13-03实验过程中使用到了IDA中的两个插件辅助分析：FindCrypt2，IDA熵插件：

[Findcrypt](#)：IDA pro插件，用于查找加密常量（以及更多）；

[IDAtropy](#)：旨在使用idapython和matplotlib的强大功能生成熵和直方图。

[IDA pro使用 findcrypt3 插件 ida findcrypt3.py-CSDN博客](#)

[danigargu/IDAtropy: IDAtropy 是 Hex-Ray 的 IDA Pro 的插件，旨在使用 idapython 和 matplotlib 的强大功能生成熵和直方图图表。](#)